

Team Members:

Qingyi Tian • Yining Shen • Chih-Hsing Hsieh

CS6650 Assignment 5

Scaling E-Commerce: A Distributed Microservices Architecture

The Challenge

- **How do you build an online store** that doesn't crash when 2,000 people shop at the same time?
- **Requirements:**
 - Browse thousands of products instantly
 - "Add to Cart" feels instant
 - Checkout works reliably
 - Scale automatically when traffic spikes
- **The twist:** These requirements conflict. Fast reads $\neq$ Fast writes. We had to make trade-offs.

01

**System
Architecture**

02

**Database
Design
Decisions**

03

**Load Testing
Setup &
Results**

04

**Autoscaling &
Bottleneck
Analysis**

05

**Lessons
Learned**

Table of contents

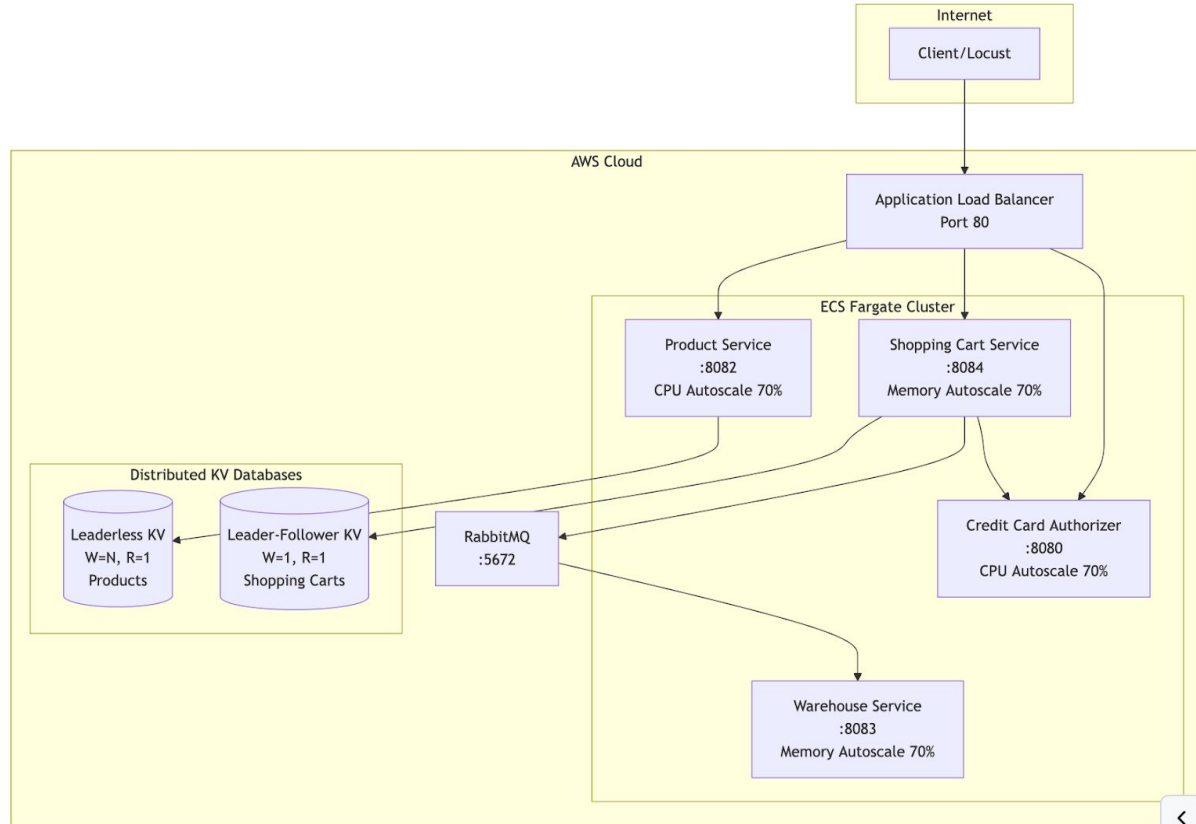
01

System

Architecture

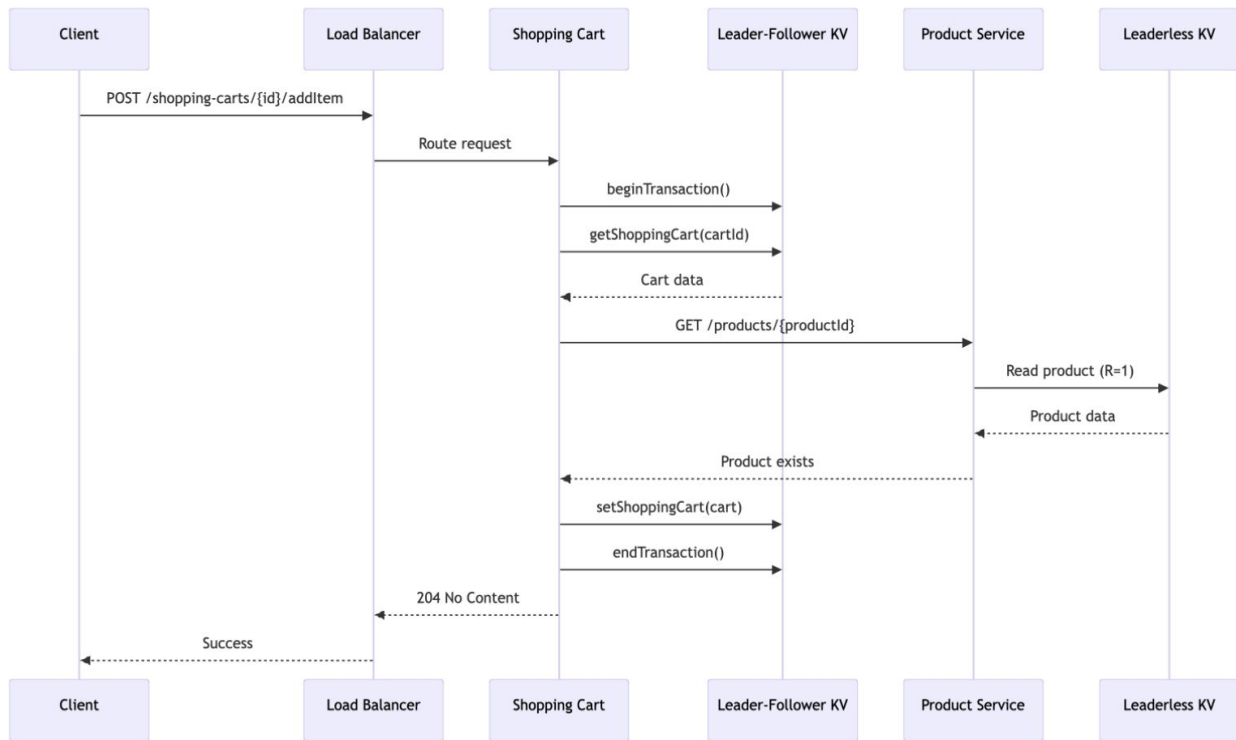
Four Services, Two Databases, One Goal

Product Service → Leaderless KV
Shopping Cart → Leader-Follower KV
Warehouse → RabbitMQ (async)
All behind ALB with autoscaling



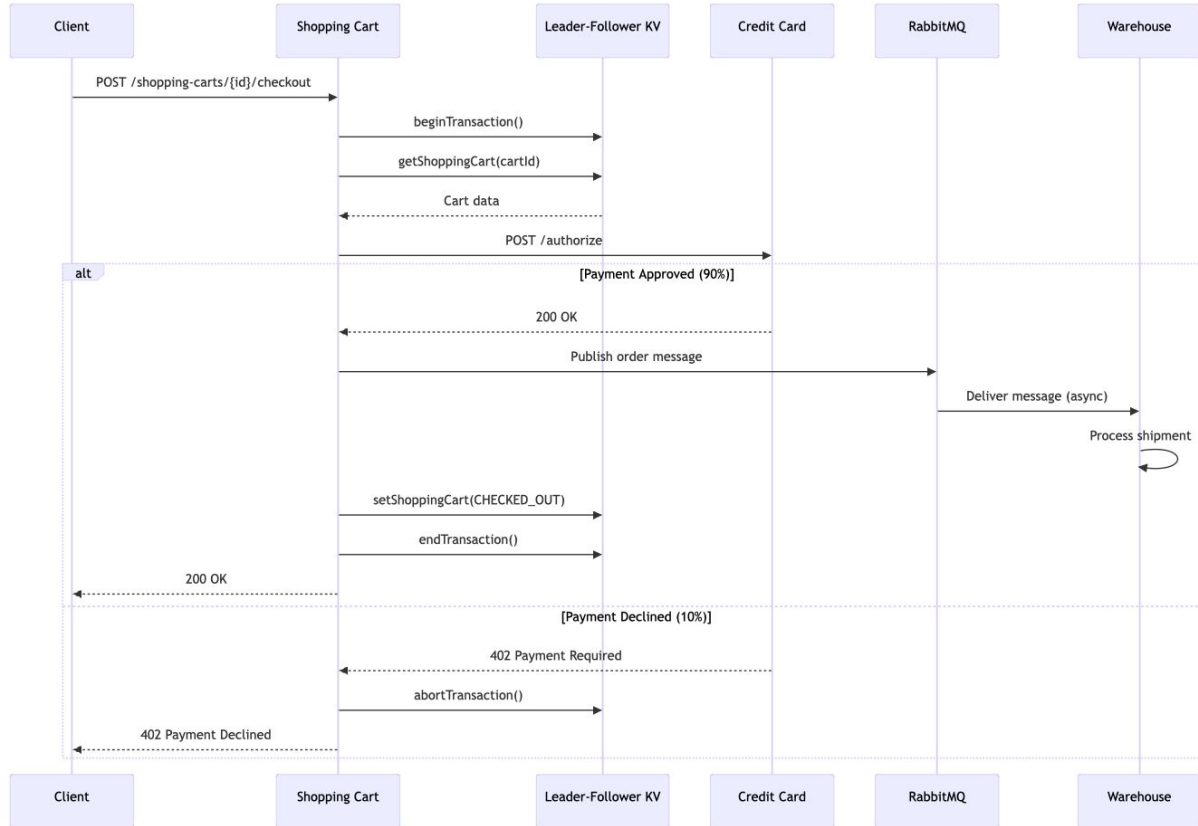
User Case -Add Item to Cart

The Shopping Cart Service acts as orchestrator: it retrieves the current cart state from the Leader-Follower KV, validates the product exists by calling Product Service (which queries the Leaderless KV), then persists the updated cart. This flow involves 3 database operations and 1 inter-service call per request.



Use Case - Checkout

Shopping Cart orchestrates 5+ operations per request



02

The key Decision -

Database Design

Product Service - Leaderless KV

Read-Heavy Workload → $W=N, R=1$

Configuration:

- $W = N$ (write to ALL nodes)
- $R = 1$ (read from ANY node)
- 5 peer nodes, no single point of failure

Real-world analogy: Amazon's product catalog - updated once a day, read by millions every hour. Slow writes are acceptable, fast reads are critical.

Factor	Value
Read/Write	90% read, 10% write
Update frequency	Rarely (1-2 per day)
Who reads?	Everyone (millions)
Who writes	Admin team only

Product Service - Leaderless KV

Product Service (View Product) had only 1 failure out of 30,044 requests - that's a 0.003% failure rate. This is strong evidence that the Leaderless KV (W=N, R=1) design works well for read-heavy workloads.

Result

Request Statistics



Type	Name	# Requests	# Fails	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	RPS	Failures/s
POST	UC1.1 Create Cart	16407	2867	11238.15	64	48258	46.48	28.76	5.03
POST	UC1.2 Add Item	36959	6178	14222.8	63	54238	28.38	64.79	10.83
POST	UC1.3 Checkout	11990	1235	14843.43	63	55652	37.73	21.02	2.16
GET	UC2 View Product	30044	1	1220.18	1	2601	132.36	52.67	0
	Aggregated	95400	10281	9692.63	1	55652	65.41	167.24	18.02

Shopping Cart Service

- Leader-Follower KV

Write-Heavy Workload → W=1, R=1

Configuration:

- W = 1 (write to leader only, async replication)
- R = 1 (read from leader)
- Followers replicate async for backup

Real-world analogy: Your Target shopping cart - only YOU modify it, you expect instant 'Add to Cart' response. No conflicts, fast writes essential.

Factor	Value
Read/Write	40% read, 60% write
Update frequency	Multiple per session
Who reads?	Only cart owner
Who writes?	Only cart owner

Shopping Cart Service

- Leader-Follower KV

- Handled 167 RPS aggregate throughput
- Scaled 1→5 instances under load (autoscaling worked)
- 502 errors triggered scaling, then system stabilized

Product Service - Low failure rate proves Leaderless works for reads
Shopping Cart - Autoscaling response proves Leader-Follower handles write-heavy load

Result

Aggregated	95400	10281	9692.63	1	55652	65.41	167.24	18.02
------------	-------	-------	---------	---	-------	-------	--------	-------

Failures Statistics			
# Failures	Method	Name	Message
2867	POST	UC1.1 Create Cart	CatchResponseError('Failed to create cart: 502')
6178	POST	UC1.2 Add Item	CatchResponseError('Add item failed: 502')
4	POST	UC1.3 Checkout	CatchResponseError('Checkout failed: 500')
1231	POST	UC1.3 Checkout	CatchResponseError('Checkout failed: 502')

shopping-cart-service

arn:aws:ecs:us-e

Active

REPLICA

FARGATE

shopping-c...

5/5 Tasks running

Completed

View

19 hours ago

CAP Theorem Trade-off

**We Chose Availability Over
Consistency**

Choice: AP (Availability + Partition Tolerance)

WHY?

01 **Revenue loss from unavailability > cost of brief inconsistency**

If a customer can't add items to cart, they leave and shop elsewhere. A brief moment of stale data (milliseconds) won't lose a sale, but a "Service Unavailable" error will.

02 **Cart data is user-isolated (no write conflicts)**

Your cart is only accessed by you. Unlike inventory (where two users might buy the last item), cart data has one owner. No conflicts = eventual consistency is safe.

03 **Checkout reads from leader (consistency when it matters)**

When you checkout, we read from the leader to get the exact cart state. Browsing can tolerate stale data, but payment must be accurate.

03

Locust

Load Testing Setup

We Stress-Tested with 2,000 Users

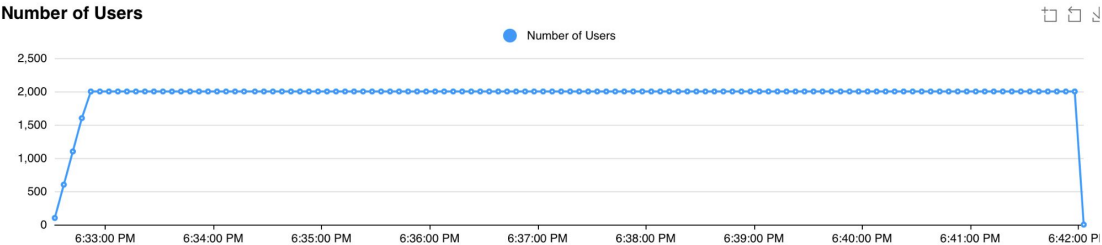
Configuration:

Parameter	Value
Tool	Locust
Users	2,000 concurrent
Duration	9.5 minutes
Target	AWS ALB

What we're measuring:

- Throughput (RPS)
- Failure rate
- Response times (50th, 95th percentile)
- Which service breaks first

Number of Users



We Stress-Tested with 2,000 Users

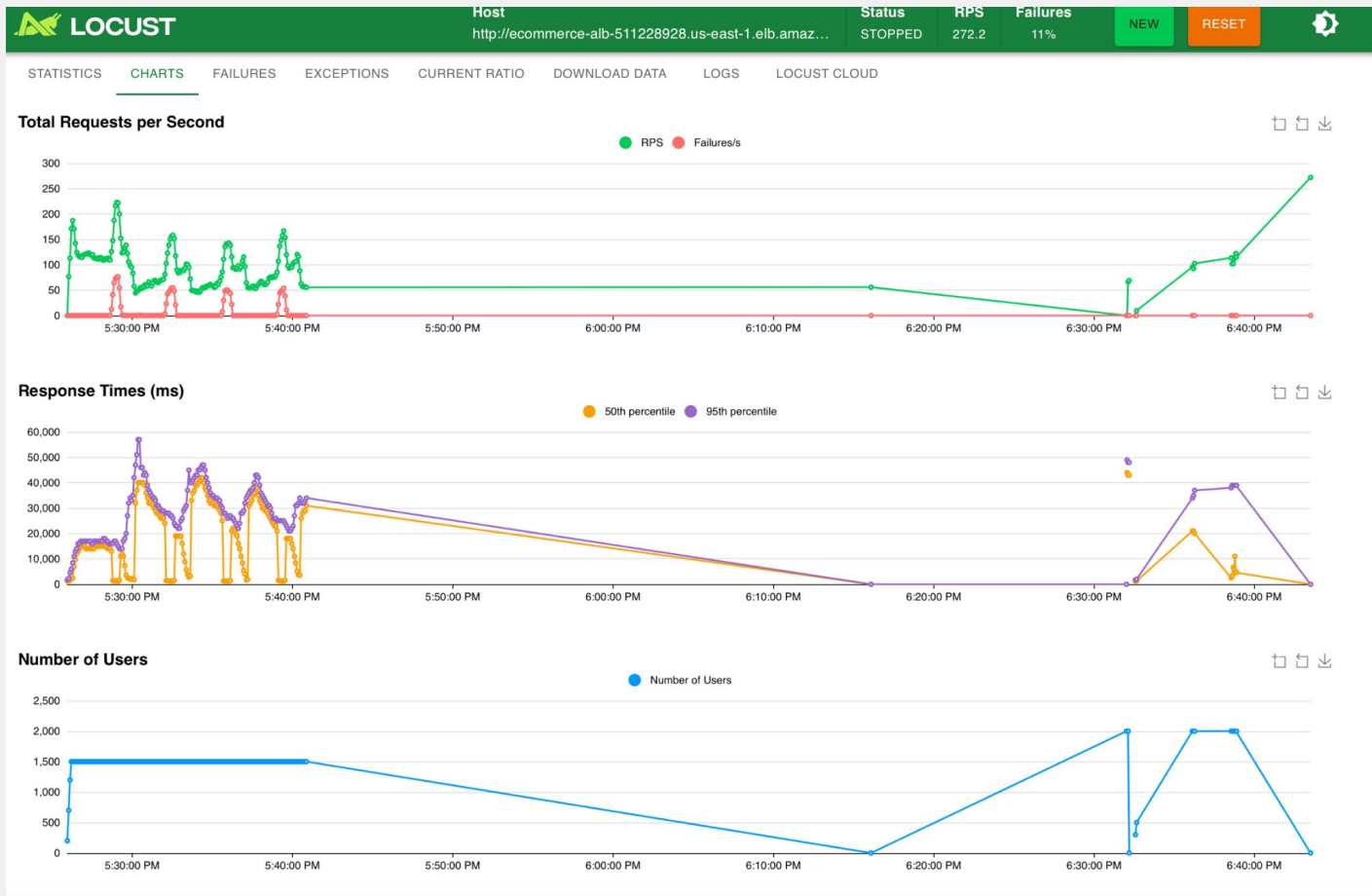
Two Use Cases:

Use Case	Weight	Flow
UC1: Shopping Session	70%	Create cart → Add items → Checkout
UC2: Product Browsing	30%	Browse products only

Why this distribution?

- **70%** shopping: Stress-test the full flow (all services)
- **30%** browsing: Simulate customers who don't buy (read-heavy)

Test Result



Request Statistics

Key Observations:

- 1. Throughput: 167.24 requests/second aggregate
- 2. Failure Rate: 10.8% (10,281 / 95,400) - primarily 502 errors from system overload
- 3. Product Service Performance: Near-zero failures (1 failure), fastest response times (~1.2s average)
- 4. Shopping Cart Bottleneck: Highest failure rates on cart operations due to service saturation

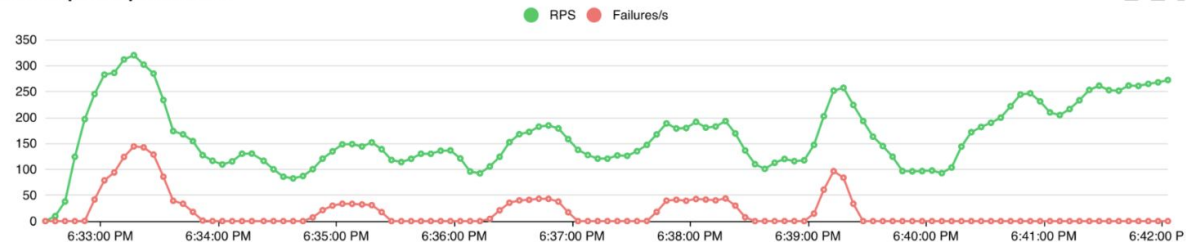
Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	UC1.1 Create Cart	16407	2867	7200	31000	35000	11238.15	64	48258	46.48	46.5	0
POST	UC1.2 Add Item	36959	6178	11000	36000	40000	14222.8	63	54238	28.38	112.4	0
POST	UC1.3 Checkout	11990	1235	11000	36000	44000	14843.43	63	55652	37.73	43.4	0
GET	UC2 View Product	30044	1	1200	1900	2000	1220.18	1	2601	132.36	69.9	0
	Aggregated	95400	10281	3700	33000	39000	9692.63	1	55652	65.41	272.2	0

Test Result

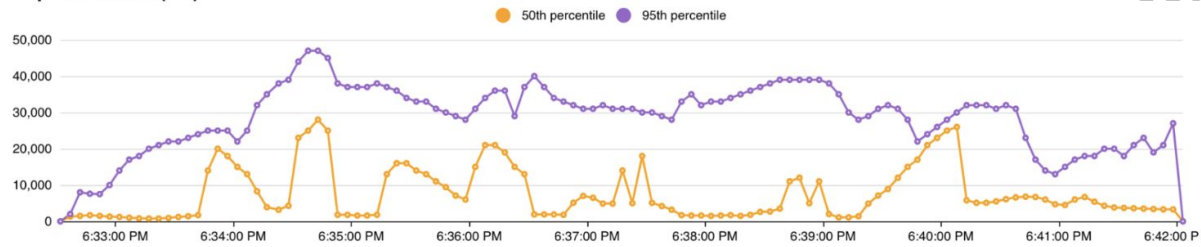
The Locust charts visualize system saturation over time: RPS stabilized around 150-250 while failure spikes (red) correlated with response time increases. As users ramped to 2,000, the 95th percentile response time reached 30,000-40,000ms. The 502 errors triggered autoscaling of the Shopping Cart Service from 1 to 5 instances.

Charts

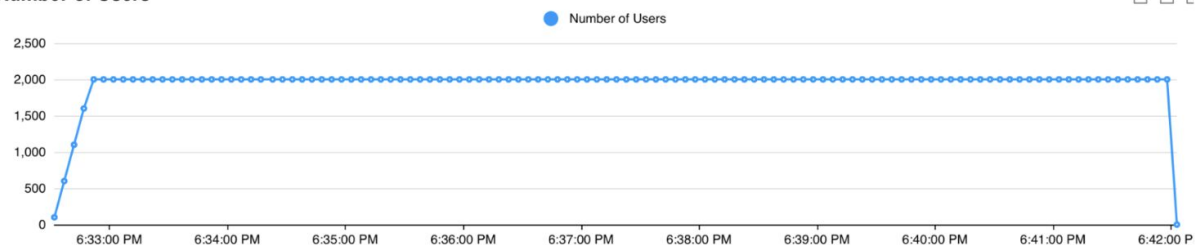
Total Requests per Second



Response Times (ms)



Number of Users



Latency Analysis - Response Time Staticitics

Method	Name	50%ile (ms)	60%ile (ms)	70%ile (ms)	80%ile (ms)	90%ile (ms)	95%ile (ms)	99%ile (ms)	100%ile (ms)
POST	UC1.1 Create Cart	7200	12000	18000	23000	28000	31000	35000	48000
POST	UC1.2 Add Item	11000	16000	22000	27000	32000	36000	40000	54000
POST	UC1.3 Checkout	11000	18000	23000	29000	33000	36000	44000	56000
GET	UC2 View Product	1200	1300	1400	1600	1700	1900	2000	2600
	Aggregated	3700	7000	13000	21000	29000	33000	39000	56000

Key Observations:

- 1. Product Service is fast and consistent: ~1.2s median with minimal variance. Leaderless KV with R=1 works well for read-heavy workloads.
- 2. Shopping Cart operations are slower: 7-11s median due to multiple service calls (KV+ Product Service + Credit Card) and service saturation under load.
- 3. Read vs Write performance gap: Product browsing is ~10x faster than cart operations,validating our database design choice (Leaderless for reads, Leader-Follower for writes).

04

Autoscaling

Autoscaling Configuration

Service	Metric	Threshold	Min	Max(Terraform)
Product Service	CPU Utilization	70%	1	3
Shopping Cart Service	Memory Utilization	70%	1	3
Credit Card Authorizer	CPU Utilization	70%	1	3
Warehouse Service	CPU Utilization	70%	1	3

Note: During load testing, we increased Shopping Cart max capacity to 5 via AWS CLI to demonstrate additional scaling headroom. We also added a CPU-based scaling policy in addition to Memory.

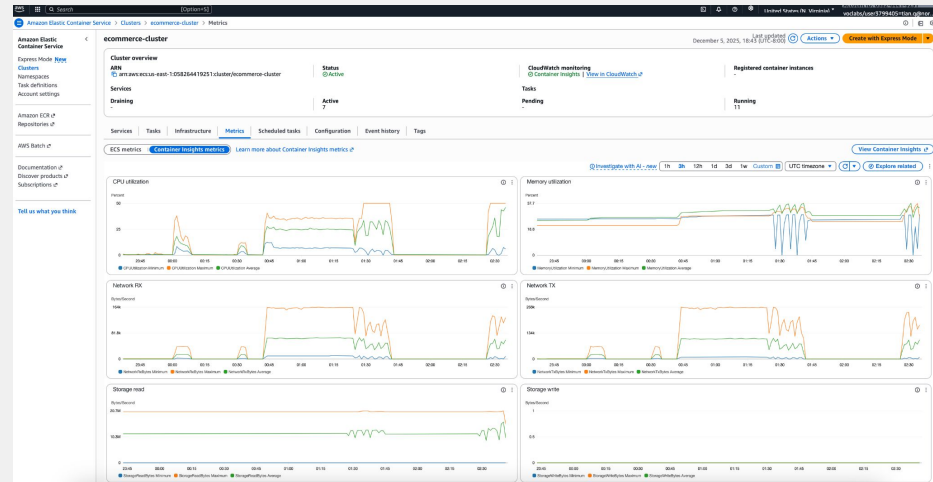
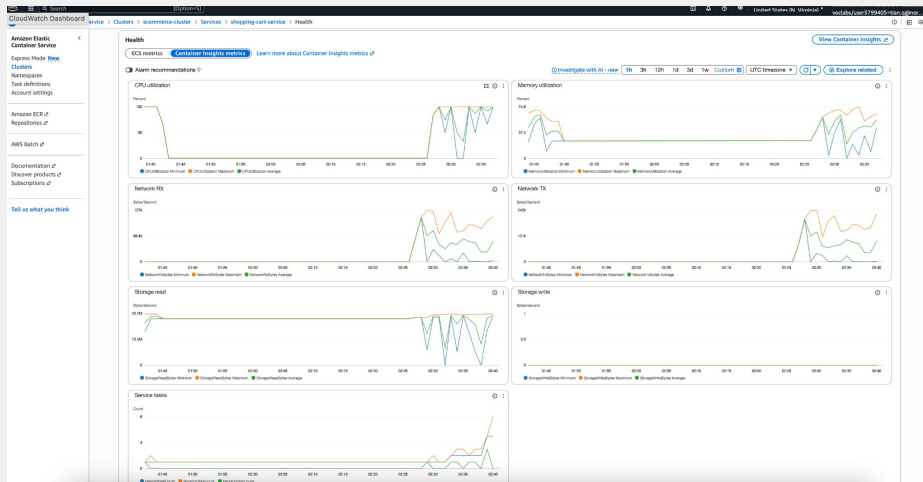
Autoscaling Evidence

CloudWatch Metrics (Shopping Cart Service):

- CPU utilization spiked to ~100% under load, triggering autoscaling
- Memory utilization remained stable at ~37% (not the bottleneck)
- Service tasks scaled to 5/5 instances (reached max capacity)

Cluster Overview:

- Total running containers: 11 (7 services active)
- Shopping Cart accounted for 5 of the 11 running tasks



Scaling Result

Service	Initial	Final	Scaled
Shoping Cart Service	1	5	Yes
Product Service	1	1	No
Credit Card Suthorizer	1	1	No
Warehouse Service	1	1	No

Why only Shopping Cart scaled: Examining the code reveals the workload imbalance:

- Shopping Cart: Each request involves business logic delay + KV read + external HTTP call (Product Service or Credit Card) + KV write + RabbitMQ publish. The addItemToCart() and checkoutCart() methods chain multiple synchronous operations.
- Product Service: Single operation per request - just business logic delay + one KV read/write.
- Credit Card / Warehouse: Stateless processing with no database calls.
- The Shopping Cart Service handles 3-5x more operations per request than other services, explaining why it alone reached capacity.

Conclusion

Decision	Rationale	Result
Leaderless KV (W=N, R=1) for Products	Read-heavy workload, rare updates	~1.2s response, near-zero failures
Leader-Follower KV (W=1, R=1) for Carts	Write-heavy workload, fast updates	Scaled to handle 167 RPS
AP over CP (CAP trade-off)	Availability critical for e-commerce	System remained responsive under load
CPU-based autoscaling for Shopping Cart	Identified as primary bottleneck	Scaled 1→5 instances automatically

Lessons Learned:

- Workload analysis drives database design - Matching replication strategy to read/write ratio is critical.
- Orchestrator services become bottlenecks - Services that coordinate multiple downstream calls require more scaling headroom.
- Autoscaling requires correct metrics - CPU was the actual constraint, not memory; monitoring data informed our configuration changes. The system successfully handled 95,400 requests at 167 RPS with 2,000 concurrent users, demonstrating that the architecture scales horizontally under load.

Q&A

Thank you